Title: Reflected XSS into HTML context Lab Report

Lab Information:

- **Lab Name: Reflected XSS into HTML context with nothing encoded**
- **Lab Difficulty Level: Apprentice**
- **Date Completed:05/11/2023**

Methodology:

1. **Initial Observation:** The lab featured a reflected cross-site scripting (XSS) vulnerability in the search functionality.
2. **Exploiting the Vulnerability:** I noticed that when I input "Mustafa" in the search box, the search term was reflected on the page. The application's back end was not sanitizing special characters, allowing for potential injection.
3. **Payload Injection:** To exploit the vulnerability, I injected JavaScript in the search parameter. For example, I used the URL `https://0ab200a604df1d94823da1d30014007b.web-security-academy.net/?search=<script>alert()</script>`
4. **Execution of Payload:** Upon submitting the payload, I observed that the injected JavaScript was executed in the user's browser, resulting in a pop-up alert. This demonstrated the ability to run arbitrary JavaScript code in the user's session, potentially compromising security.

Results:

- Successfully exploited the reflected XSS vulnerability to execute arbitrary JavaScript code in the user's browser, leading to a pop-up alert.

Security Implications:

- The lab highlighted the dangers of reflected XSS vulnerabilities, where an attacker can inject malicious scripts that execute in a user's browser.

Preventive Measures:

- To prevent reflected XSS vulnerabilities, applications should sanitize user inputs, encode data before reflecting it in responses, and implement secure coding practices.

Reflected XSS into HTML context with nothing encoded

LAB Not solved

Back to lab description »

Home

0 search results for 'Mustafa '

Search the blog... Search

< Back to Blog

Reflected XSS into HTML context with nothing encoded

LAB Not solved

Back to lab description »

Home

0 search results for '|\'

<h1>Mustafa<h1> Search

< Back to Blog

Reflected XSS into HTML context with nothing encoded

LAB Not solved

Back to lab description »

Home

0 search results for '

Mustafa

'

Search the blog... Search

< Back to Blog

Inspector Console Debugger Network Style Editor Performance Memory Storage Accessibility

Search HTML

```
<div id="academyLabHeader">…</div> overflow
<div theme="blog">
  <section class="maincontainer">
    <div class="container is-page"> overflow
      <header class="navigation-header">…</header> flex
      <header class="notification-header">…</header>
      <section class="blog-header">
        <h1>0 search results for '</h1>
        <h1>Mustafa</h1>
        <h1>'</h1>
        <hr>
      </section>
      <section class="search">…</section>
      <section class="blog-list no-results">…</section>
    </div>
  </section>
  <div class="footer-wrapper">…</div> overflow
```

html > body > div > section.maincontainer > div.container.is-page > section.blog-header > h1

https://0ab200a604df1d94823da1d30014007b.web-security-academy.net/?search=<script>alert()</script>

king.com Full time Jobs in Helsi... id Poe - Fast, Helpful AI ... network.drawio.html - ... Untitled Diagram.dra... PayloadsAllTheThings/...

0ab200a604df1d94823da1d30014007b.web-security-academy.net

OK

Reflected XSS into HTML context with nothing encoded

LAB Solved

Back to lab description »

ou solved the lab! Share your skills! Continue learning »

Home

0 search results for ''

Search the blog... Search

< Back to Blog

Title: Stored XSS into HTML Context Lab Report

Lab Information:

- Lab Name: Stored XSS into HTML context with nothing encoded
- **Lab Difficulty Level:** Apprentice
- **Date Completed**: 05/11/2023

Methodology:

5. **Initial Observation:** The lab featured a stored cross-site scripting (XSS) vulnerability in the comment functionality.
6. **Testing Input Filtering:** I initially tested the comment section for input filtering. I entered special characters, including `<h1>My Name Mustafa</h1>`, to see if the backend would filter and encode the input upon reflection.
7. **Identifying Lack of Filtering:** To my surprise, the backend did not filter or encode the HTML tags, indicating a potential vulnerability in the application.
8. **Exploiting the Vulnerability:** I then injected JavaScript in the comment section as `<script>alert()</script>`. As this content was stored in the database, it could be executed when other users viewed the blog post with my comment.
9. **Execution of Payload:** When I revisited the home page, the JavaScript alert popped up, demonstrating the successful execution of the payload. This meant that any user visiting the site could become a victim of this attack.

Results:

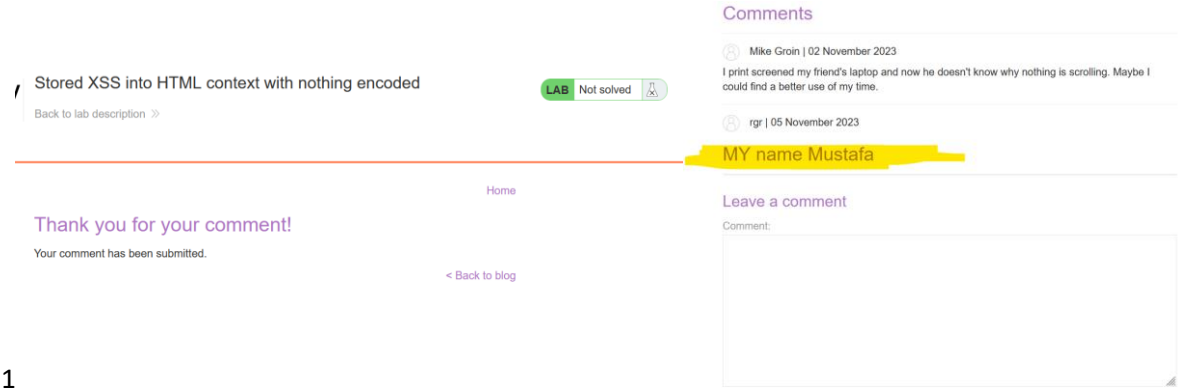
- Successfully exploited the stored XSS vulnerability by injecting and executing JavaScript code in the comment section.
- Demonstrated the risk of unfiltered and unencoded user inputs in web applications.

Security Implications:

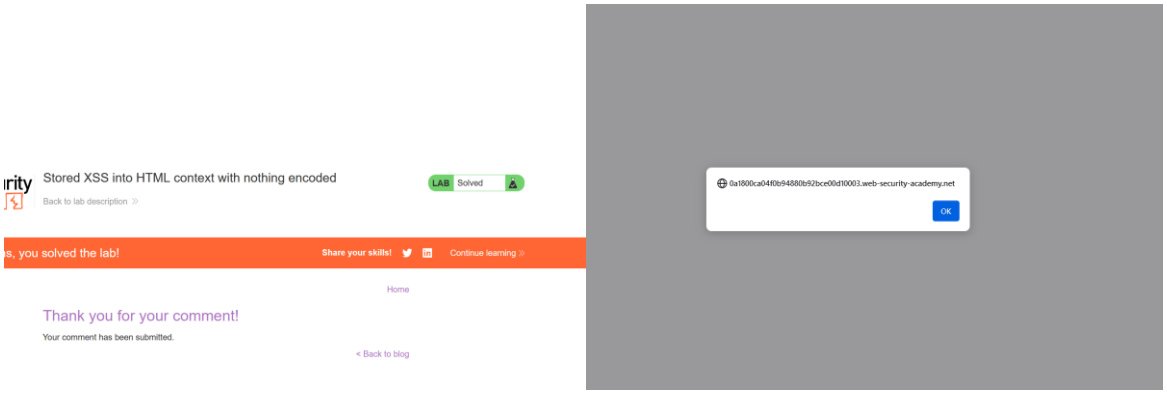
- The lab highlighted the dangers of stored XSS vulnerabilities, where malicious code can persist and affect other users who view the content.

Preventive Measures:

- To prevent stored XSS vulnerabilities, applications should implement input filtering, data encoding, and security best practices.



1



.....

Title: DOM XSS in document.write Sink Lab Report

Lab Information:

- Lab Name: DOM XSS in document.write sink using source location.search
- Lab Difficulty Level: Apprentice

Methodology:

1-Initial Observation: The lab featured a DOM-based cross-site scripting (XSS) vulnerability in the search query tracking functionality, which utilized the `document.write` function.

2-Analyzing the Source Code: I examined the source code and noticed an `img` tag with the `src` attribute set to a path that included a `searchTerms`

parameter. For example: `img src="/resources/images/tracker.gif?searchTerms=vtgtgcf"`.
3-Exploiting the Vulnerability: I realized that once this image was loaded, it triggered the execution of the `alert` function. To exploit this, I injected a payload in the URL. I included a double quote in the search query to break out of the attribute, and then I started looking for a new attribute that I could use for the attack.
4-Payload Injection: I crafted the payload as follows: `vtgtgcf||" onload="alert()` in the URL parameter, which injected an `onload` attribute into the `img` tag. This attribute triggered the `alert()` function when the image was loaded.
5-Execution of Payload: When I revisited the page, the JavaScript `alert()` function popped up, indicating the successful execution of the XSS attack.
Results:

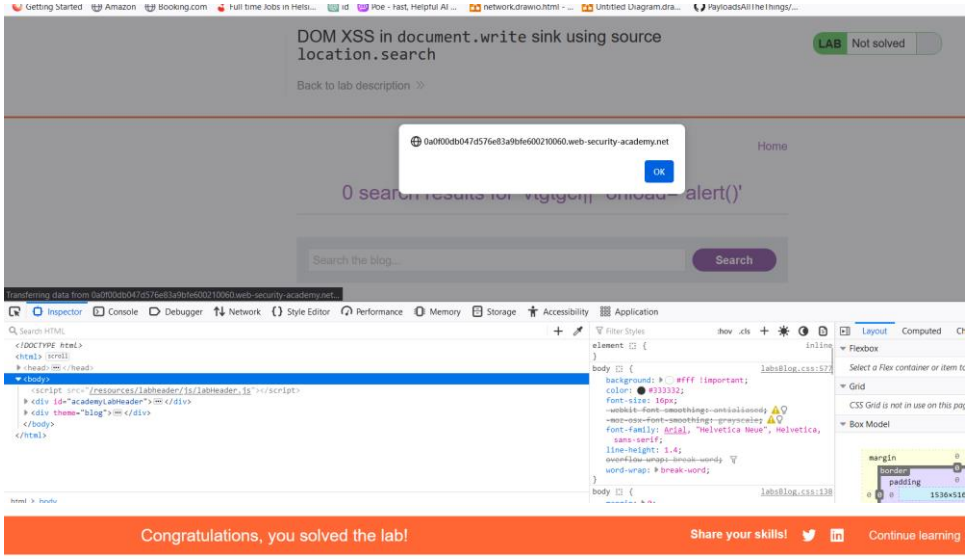
- Successfully exploited the DOM-based XSS vulnerability using the `document.write` function and `location.search` to trigger the `alert` function.
- Demonstrated the danger of DOM-based XSS and the need for proper input validation.

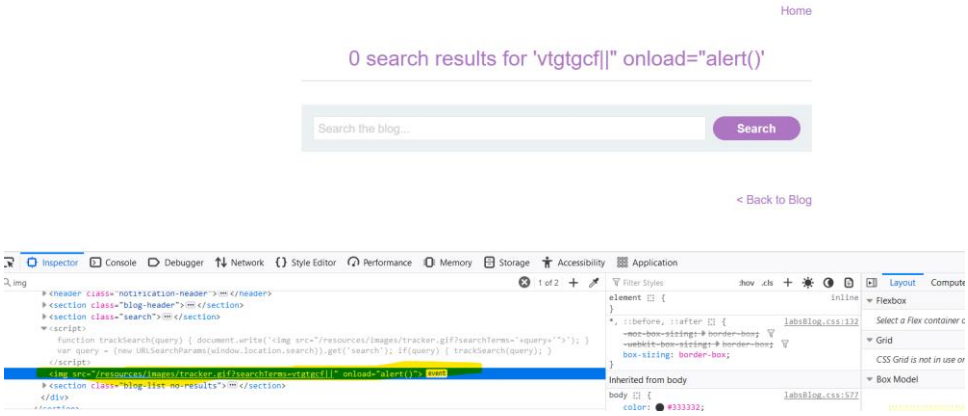
Security Implications:

- The lab highlighted the risk of DOM-based XSS vulnerabilities, which can be exploited to execute malicious scripts in the context of a user's browser.

Preventive Measures:

- To prevent DOM-based XSS vulnerabilities, applications should validate and sanitize user inputs, and avoid using user-controlled data in JavaScript functions without proper encoding.


DOM XSS in document.write sink using source location.search
LAB Not solved
Back to lab description
0a0f00db047d576e83a9bfe600210060.web-security-academy.net
OK
Home
Search the blog...
Search
Congratulations, you solved the lab!
Share your skills!
Continue learning


Home
0 search results for 'vtgtgcf||" onload="alert()'
Search the blog...
Search
< Back to Blog